Logical database design

- The learning objectives for this week are:
  - Knowing what is the objective **logical database design**
  - Knowing how to **derive relations** from entity-relationship diagrams
  - Knowing how to determine **natural primary keys** and **foreign keys**

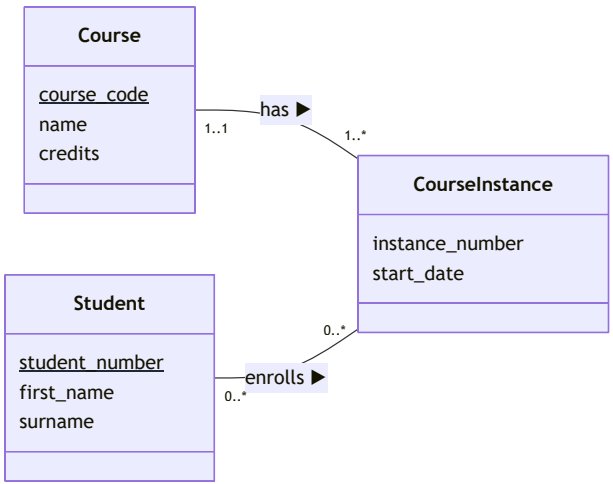
A substantial portion of these materials is derived from the work of Kari Silpiö. Any use, reproduction, or distribution of this content requires prior written permission from him.

Logical database design

- The typical main phases in a database design process are:
  1. Conceptual database design
  2. Logical database design
  3. Physical database design
- Each phase from top to bottom adds more detail to the design
- We have familiarized ourselves with the **conceptual database design** by defining entities and their attributes and relationships based on the requirements
- The **logical database design** is the process of refining and translating the conceptual schema into a **logical database schema** based on a specific data model, for example the relational model

Conceptual vs. logical database schema

- Here's a comparison of **conceptual** (left) and **logical** (right) database schema of a course enrollment database:




```
Course(course_code, name, credits)
Student(student_number, first_name, surname)

CourseInstance(course_code, instance_number, start_date)
  FOREIGN KEY (course_code) REFERENCES Course(course_code)

Enrollment(course_code, instance_number, student_number)
  FOREIGN KEY (course) REFERENCES Course(course_code)
  FOREIGN KEY (course_code, instance_number)
  REFERENCES CourseInstance(course_code, instance_number)
  FOREIGN KEY (student_number)
  REFERENCES Student(student_number)
```

Logical database design

- Typically, the logical database design process includes the following types of activities:
  1. Create relations from conceptual schema's entity types by following a fairly simple process
  2. Validate that the relations don't have redundancies using normalization techniques
  3. Validate relations against users' requirements. For example, *"list all course names a student has enrolled to"* could be a user's requirement for a schema
  4. Verify integrity constraints, e.g., primary keys, foreign keys and not null contraints
  5. Review the model with users

Deriving relations for the logical data model

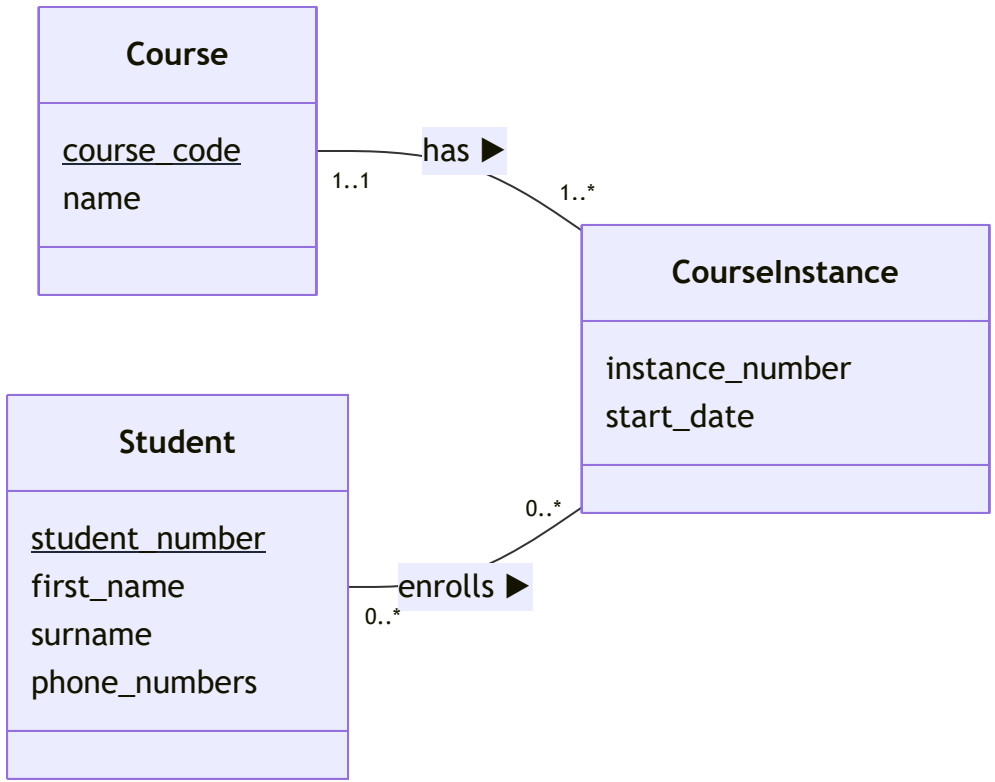
- The process starts by **creating relations for the logical data model**, which includes:
  1. Creating the relations from the entitity types
  2. Refining the attributes. For example, a non-atomic `full_name` attribute could be refined to `first_name` and `surname` attributes
  3. Determining primary and foreign keys
  4. Determining other types of integrity constraints. For example `Course` relation's `name` attribute should have a not null constraint

Creating relations

- To transform a conceptual schema into a relational schema, we might need **additional relations** to establish specific relationships
- We create the relations in the following manner:
  - For **each entity type**, we create a relation that includes all simple (single-value) attributes of the entity
  - For **many-to-many relationship types** ("…*" on both sides of the relationship), we create a **"bridge relation"** to represent the relationship
  - For **multi-valued attributes**, we create a new relation to represent the multi-valued attribute. For example, a person may have several phone numbers, but multi-valued attributes are not allowed in relations

Example of creating relations

- Let’s consider creating relations for the following conceptual model:



Example of creating relations

- In the example we have one **many-to-many relationship** between the `Student` and `CourseInstance` entities ("…*" on both sides of the "enrolls" relationship):
  - *"Student enrolls to zero or many course instances and course instance has zero or many students"*
  - In this case we create an additional **bridge relation**, for example `Enrollment`
- There is a **multi-valued attribute** `phone_numbers` in the `Student` entity
  - In this case we create an additional entity, for example `StudentPhoneNumber`
- This leaves with the following relations: `Student`, `Course`, `CourseInstance`, `Enrollment` and `StudentPhoneNumber`

Refining the attributes

- Once we have created the relations, we need to refine the attributes in the following manner:
  - Divide non-atomic attributes into smaller, atomic attributes. For example, a student's home address can be split into `city`, `postal_code`, and `street_address`
  - Define general (not DBMS-specific) attribute data types, such as `text`, `integer` or `date`
  - Decide which attributes can have a `NULL` (missing) value. Allow a `NULL` value only when there is a clear and valid reason, because inappropriate `NULL` values degrade data quality

Determining primary keys

```
Student(student_number, first_name, surname)
```

- There should be **exactly one primary key in each relation**
- The primary key can be either a **simple key** (single column, like `course_code` in the `Course` relation) or a **composite key** (several columns, like `course_code` and `instance_number` in the `CourseInstance` relation)
- By definition, the primary key should always satisfy the properties of **requiredness** (not `NULL`), **uniqueness** and **minimality**
- The primary key **should remain stable**. That is, primary key values should not be updated later
- The primary key should be **reasonably compact** (e.g. a short string or an integer), to improve performance and minimize storage

Primary key of a weak entity type

- A **weak entity type** depends on other entity types and cannot be uniquely identified by its own attributes alone
- For example, a `CourseGrade` cannot exist without a `Student` and a `CourseInstance`.
- When creating a relation from a weak entity type, its primary key includes the primary keys of its owner relations
- For example, the `CourseGrade` relation uses `student_number`, `course_code`, and `instance_number` as its primary key. These attributes come from the `Student` and `CourseInstance` relations
- In such case, we need to define the foreign keys first, then use them to define the weak entity relation's primary key

Surrogate keys

- If there is initially no candidate key available for a relation, then we cannot determine a **natural primary key**
- In this case, we **add a new attribute** with a unique value to serve as the primary key, called a **surrogate key**
- Surrogate keys are commonly system-generated values, such as random values or incrementing numbers, like the `messageid` primary key in the data below

messageid	from	to	title	body
1	kalle.ilves@haaga-helia.fi	john.doe@gmail.com	Greeting	Hello John!
2	john.doe@gmail.com	kalle.ilves@haaga-helia.fi	Response	Hello Kalle!
...	...	...	...	...

Alternate keys

- Candidate keys that are not selected to be primary the key are called **alternate keys**
- Alternate keys are unique by nature, so we should consider the use of the **unique constraint** on alternate keys to enforce their uniqueness:

```
Student (student_number, ssn, familyname, givenname)
  UNIQUE (ssn)
```

- Especially, when we are using a surrogate primary key, a unique constraint on at least one natural alternate key improves data quality

Determining foreign keys

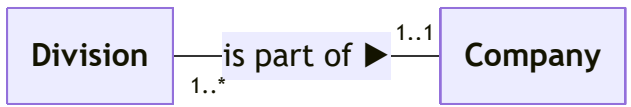
- In the relational model, relationships are represented by a **foreign key that references a primary key**
- To decide where the foreign key should be placed, we first identify the **relationship type** between the entities
- In this course, we describe foreign keys in the **relation schema** using the following notation:

```
Course(course_code, name, credits)

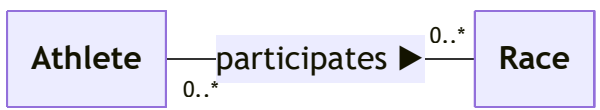
CourseInstance(course_code, instance_number, start_date)
  FOREIGN KEY (course_code) REFERENCES Course(course_code)
```

Relationship types

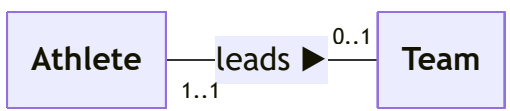
- Most often the relationship type between two entities falls in **many-to-one**, **many-to-many** and **one-to-one** categories
- Example of **many-to-one relationship** with "...*" on one side and "...1" on another:



- Example of **many-to-many relationship** with "...*" on both sides:

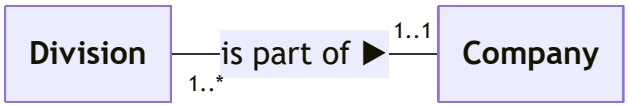


- Example of **one-to-one relationship** with "...1" on both sides:



Many-to-one relationship (1:N)

> *"Division is part of exactly one company and company has one or many divisions"*



- Place a copy of the **parent relation's** ("...1" side) primary key into the **child relation** ("...*" side), to act as a foreign key
- If the child relation is derived from a weak entity type, then the primary key of the child relation is typically a composite key

```
Company (company_id, name)

Division (division_id, name, company_id)
  FOREIGN KEY (company_id) REFERENCES Company(company_id)
```

Many-to-many relationship (M:N)

> "Athlete participates in zero or many races and race has zero or many athlete participants"

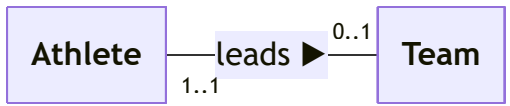


- Create a **bridge relation** to represent the relationship and place a copy of the primary key from each of the parent relations into the bridge relation to act as foreign keys
- Typically, the bridge relation's primary key is a composite key that includes the both foreign keys

```
Athlete (athlete_id, first_name, family_name)
Race (race_id, name, date)
RaceParticipation(athlete_id, race_id)
  FOREIGN KEY (athlete_id) REFERENCES Athlete(athlete_id),
  FOREIGN KEY (race_id) REFERENCES Race(race_id)
```

One-to-one relationship (1:1)

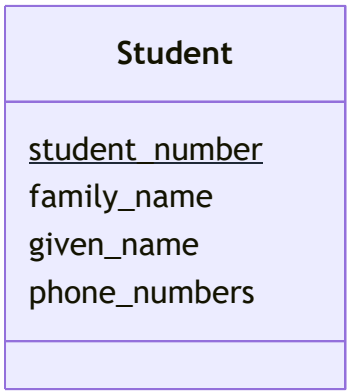
> "Team has exactly one leader and athlete leads zero or one team"



- In case of **mandatory participation** ("1…1" on one side only), place a copy of the primary key from the relation on the "1…1" side into the relation on the "0…1" side to act as the foreign key
- In case of **mandatory participation on both sides** we can usually **combine the two** into one relation
- In case **optional participation** ("0…1" on both sides) the foreign key can be placed in either relation

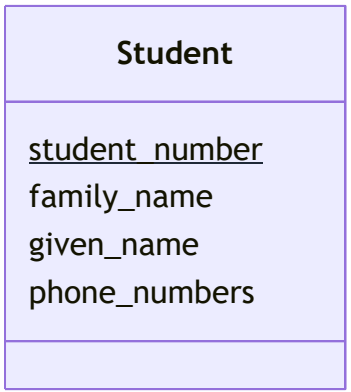
```
Athlete (athlete_id, first_name, family_name)
Team (team_id, name, athlete_id)
  FOREIGN KEY (athlete_id) REFERENCES Athlete(athlete_id)
```

Multi-value attributes



- A relation can't have attributes with **multiple values**, such as the `phone_numbers` attribute of the `Student` entity type in this example (student has many phone numbers)
- Instead, we replace the multi-value attribute with a **new relation**, such as `StudentPhone` relation
- The multi-value attribute (`phone_numbers`) will be removed from the original relation and placed to the new relation
- The original relation's primary key (`student_number`) is added as a foreign key to the new relation
- The primary key of the new relation is commonly a **composite key** consisting of the attribute and the foreign key (`phone_number, student_number`)

Multi-value attributes



- In this example, we would get the following relation schema:

```
Student (student_number, first_name, family_name)
StudentPhone (phone_number, student_number)
  FOREIGN KEY (student_number) REFERENCES Student(student_number)
```

Summary

- The objective of logical database design is to translate the conceptual schema into a **logical database schema** based on a specific data model
- During the course, we represent logical database schema as **relation schema**, containing relations their attributes, primary key and foreign keys
- When we derive relations from entity types, we create a relation for each entity type
- Many-to-many relationship requires an additional **bridge relation**
- A **multi-valued attributes** requires an additional relation
- There should be exactly one primary key in each relation
- The foreign key placement depends on the **relationship type** (many-to-one, many-to-many or one-to-one)